SpendWise AI Studio — Ultimate Interview Question Bank

This document is an exhaustive, professional interview preparation guide tailored specifically for the **SpendWise AI Studio** project. It covers **System Architecture**, **Multi-Agent Systems (CrewAI & Gemini)**, **OCR & Data Ingestion**, **Frontend Engineering (Streamlit)**, **Prompt Engineering & Security**, **Scalability**, and **Behavioral Storytelling**.

■ Section 1: System Architecture & High-Level Design

Q1: What is SpendWise AI Studio, and what core problem does it solve?

Answer / Key Points:

- **The Problem:** Traditional personal finance apps rely on manual expense entry or basic bank syncs that lack line-item granularity. College students and young professionals often struggle to understand where their money goes at the individual item level (e.g., distinguishing between buying snacks vs. real groceries at a supermarket).
- **The Solution:** SpendWise is an enterprise-grade financial audit and budgeting platform. It combines **OCR (Optical Character Recognition)** to extract itemized line items from raw receipt images, orchestrates a **3-agent AI Crew (CrewAI + Google Gemini)** to intelligently categorize expenses and detect spending anomalies, and delivers personalized budgeting advice through a **custom, glassmorphic Streamlit web studio**.

Q2: Can you explain the end-to-end data pipeline of SpendWise?

Answer / Key Points:

1. **Ingestion & OCR (scan-reciepts-hf.py / ocr_space_receipts.py):** Receipt images (**.jpg**, **.png**) in the **receipts/** directory are processed via Asprise OCR or OCR.space API. Raw text/tables are parsed into normalized line items (**item_id**, **description**, **quantity**, **amount**) and saved to **outputs/ocr_results.json**.
2. **AI Audit & Categorization (run.py & agents_And_tasks.py):**

- Reads **ocr_results.json**.
- Checks **categorized_receipts.json** for cached items to prevent redundant LLM API calls.
- Sends unclassified items in batches of 20 to the **Categorizer Agent**.
- Calculates overall and category totals using deterministic Python math.
- Passes summaries to the **Analyst Agent** (trend observation) and **Advisor Agent** (student budgeting strategies).
- Output is saved to **outputs/crew_analysis.json**.

3. **Visualization & Interaction (app.py & analysis.py):**

- The Streamlit dashboard (**app.py**) loads the JSON reports, renders interactive Plotly charts, displays KPI badges, and hosts a live conversational AI financial coach.

Q3: Why did you choose a decoupled, file-based JSON architecture over a direct database or monolithic script?

Answer / Key Points:

- **Modularity & Debuggability:** By separating OCR ingestion (**scan-reciepts-hf.py**), AI processing (**run.py**), and UI presentation (**app.py**) via intermediate JSON files in **outputs/**, each subsystem can be developed, tested, and debugged independently.
- **Caching & Cost Optimization:** Storing **categorized_receipts.json** acts as a local persistence layer. If the user re-runs the pipeline or adds one new receipt, previously categorized items are loaded instantly from disk rather than re-billing the Gemini LLM API.
- **Simple Deployment:** For a student prototype or MVP, JSON files eliminate the setup friction and connection overhead of an external relational database (like PostgreSQL) while maintaining clean schema structure.

Section 2: AI & Multi-Agent Systems (CrewAI & Gemini)

Q4: Why did you use a Multi-Agent framework (CrewAI) instead of a single LLM prompt?

Answer / Key Points:

- **Separation of Concerns & Role Specialization:** In a single prompt, asking an LLM to simultaneously categorize 100 items, compute math, analyze trends, and format budgeting advice leads to cognitive overload, hallucinated numbers, and degraded reasoning.
- **Agent Roles in SpendWise:**

1. **Categorizer Agent:** Focuses strictly on semantic classification of item descriptions into 11 discrete buckets.
2. **Analyst Agent:** Receives aggregated math and focuses solely on high-level pattern recognition and spending concentration.
3. **Advisor Agent:** Takes the analyst's observations and crafts empathetic, actionable financial tips for college students.

- **Modular Prompting:** Each agent has a dedicated **role**, **goal**, and **backstory**, ensuring higher accuracy and domain consistency.

Q5: Why did you configure Process.sequential in CrewAI? When would you use hierarchical or parallel processing?

Answer / Key Points:

- **Sequential Dependency:** The SpendWise audit workflow is strictly linear: you *cannot* analyze spending patterns until items are categorized, and you *cannot* give personalized budgeting advice until the patterns are analyzed. Therefore, **Process.sequential** is the exact right fit.
- **When to use Parallel/Hierarchical:**
- *Parallel:* If we had independent tasks (e.g., fetching stock market news while simultaneously OCR-scanning receipts), we could execute them asynchronously.
- *Hierarchical:* If we had a complex organization of 10+ agents where a "Manager Agent" dynamically delegates tasks to junior auditors based on receipt complexity.

Q6: How do you handle LLM hallucinations or formatting errors when expecting structured JSON?

Answer / Key Points:

- **Prompt Guardrails:** In **create_categorization_task**, explicit instructions dictate: *"Return ONLY a valid JSON dictionary... Do NOT default everything to Groceries"*.
- **Post-Processing Cleanliness (clean_json_response in run.py):**
- LLMs often wrap JSON in markdown code blocks (`json ... `) or append conversational preamble.
- **clean_json_response()** uses regex substitution to strip markdown fences and string slicing (**cleaned.find("{")** to **cleaned.rfind("}") + 1**) to extract just the valid JSON object before passing it to **json.loads()**.
- **Deterministic Math:** We *never* ask the LLM to calculate total spending sums. We use deterministic Python loops over the extracted amounts (**total_spent += cost**), eliminating mathematical hallucinations.

Q7: Why do you categorize items in batches of 20 (batch_size = 20) in run.py?

Answer / Key Points:

- **Context Window & Attention Limits:** Sending 500 line items in a single prompt increases the risk that the LLM will skip items, lose attention in the middle, or exceed output token limits.
- **Latency & Retry Safety:** If an API call fails or times out on a batch of 20, we only retry those 20 items rather than re-running the entire dataset.
- **Structured Mapping:** Batching ensures every item ID (**r1_i1**, **r1_i2**) is reliably mapped and accounted for in the response dictionary.

■ Section 3: OCR & Document Ingestion

Q8: Compare the two OCR approaches in your project: Asprise OCR (scan-reciepts-hf.py) vs. OCR.space (ocr_space_receipts.py).

Answer / Key Points:

Feature	Asprise OCR (scan-reciepts-hf.py)	OCR.space (ocr_space_receipts.py)
Output Format	Structured JSON with pre-parsed fields (**merchant_name**, **date**, **items**, **total**).	Raw text strings or line-by-line parsed text (**isTable=True**).
Categorization	Delivers raw items to AI agents (CrewAI + Gemini) for semantic classification.	Uses procedural keyword matching against an internal dictionary (**CATEGORIES**).
Best Used For	Production pipeline where complex receipts require automated structural parsing.	Lightweight fallback, testing, or environments without LLM API access.

Q9: In ocr_space_receipts.py, why did you use isTable=True and OCREngine=2?

Answer / Key Points:

- **isTable=True:** Instructs the OCR engine to preserve horizontal spatial alignment. Receipts are tabular (Description on the left, Price on the right); without table mode, prices often get detached from their corresponding items.
- **OCREngine=2:** In OCR.space, Engine 2 is specifically optimized for number recognition, special symbols (currency signs like **Rs.** , **$**), and structured receipt/invoice layouts, whereas Engine 1 is tuned for blocks of standard natural language text.

Q10: How do you handle network failures and API Rate Limits (HTTP 429) during batch image processing?

Answer / Key Points:

- **Retry Loop with Exponential/Linear Backoff:** In **ReceiptCaptureService.extract_document()**, a retry loop (**max_retries = 4**) checks the HTTP status code.
- **Handling HTTP 429:** When a rate limit is detected (**status_code == 429**), the script pauses using **time.sleep(attempt * 5)** before retrying, preventing API IP bans and ensuring job completion.
- **Polite Polling:** Between successful batch captures, an intentional pause (**time.sleep(2.5)**) is executed to maintain polite throughput.

Q11: What happens if the OCR engine fails to extract individual line items from a receipt?

Answer / Key Points:

- **Graceful Fallback (transform_receipt_payload):** If **raw_items** is empty but the document has a valid total amount (**doc_total > 0**), the system automatically synthesizes a single aggregate item: **{"description": "General Purchase at {merchant_name}", "amount": doc_total}**. This ensures the user's total financial expenditure remains 100% accurate even if line-item reading fails.

Section 4: Frontend Engineering & Streamlit Studio (app.py)

Q12: How did you transform default Streamlit into a modern, non-traditional web app?

Answer / Key Points:

- **CSS Injection (apply_custom_styles):** Used **st.markdown(..., unsafe_allow_html=True)** to override default Streamlit DOM classes.
- **Typography & Chromeless UI:** Imported Google font **'Outfit'**, hid default Streamlit headers, footers, and hamburger menus (**#MainMenu {visibility: hidden;}**).
- **Glassmorphism & Gradients:** Styled the navigation bar and KPI cards with semi-transparent dark gradients (**rgba(30, 41, 59, 0.8)**), subtle borders, and backdrop blur filters (**backdrop-filter: blur(10px)**).
- **Interactive Micro-animations:** Added hover transformations (**transform: translateY(-3px)**) and glowing border transitions to KPI cards.

Q13: How do you optimize frontend performance and avoid re-reading files on every UI interaction?

Answer / Key Points:

- **Streamlit Caching (@st.cache_data):** The load_spendwise_data() function is decorated with @st.cache_data. When the user switches tabs, filters the interactive data table, or interacts with charts, Streamlit serves the cached Pandas DataFrame and JSON objects from memory instantaneously.
- **Cache Invalidation:** When the user clicks **"Run Pipeline Now"** from the UI, we explicitly call st.cache_data.clear() and st.rerun() so the new audit results are reflected immediately.

Q14: How is the floating AI Chatbot widget implemented in app.py?

Answer / Key Points:

- **UI Placement:** Used Streamlit's native st.popover(" AI Assistant") combined with custom CSS targeting div[data-testid="stPopover"] to fix its position to the bottom-right corner (position: fixed; bottom: 24px; right: 24px; z-index: 99999;).
- **Session State:** Uses st.session_state.chat_messages to maintain conversation history across re-renders without losing context.
- **Context Injection:** When calling Gemini, the chatbot injects a condensed string of the user's audited report (total_spent, top category, insights) into the system instructions, allowing the LLM to give personalized answers based on actual spending data.

Q15: In app.py, how do you trigger backend Python scripts, and what are the concurrency considerations?

Answer / Key Points:

- **Execution Mechanism:** When the user clicks "Run Pipeline Now", the app executes subprocess.run(["python", "run.py"]) inside a st.spinner() block.
- **Concurrency & Production Considerations:** In a single-user local prototype, synchronous subprocess.run works fine. In a multi-user production deployment, blocking the Streamlit thread would freeze the app for other users. To scale, we would replace subprocess with an asynchronous background job queue (e.g., Celery, Redis Queue, or FastAPI background tasks) and poll for completion via WebSockets or progress bars.

Section 5: Prompt Engineering, Guardrails & Security

Q16: How did you design the prompts for your AI agents to ensure high-quality financial advice?

Answer / Key Points:

- **Role-Goal-Backstory Paradigm:** By giving each agent a distinct persona (e.g., *"friendly personal finance mentor specializing in college student budgeting"*), the LLM adopts an appropriate tone and vocabulary.
- **Structured Constraints:** In create_advisory_task, we explicitly specify the JSON schema we expect. We also mandate actionable components: a "quick_win" (immediate low-effort action), "tips" (long-term strategies), and a "positive_note" (empathetic reinforcement).

Q17: What security guardrails are built into the Gemini Chatbot in app.py?

Answer / Key Points:

- **Strict Domain Boundary:** System instructions mandate: *"You MUST strictly answer questions related to personal finance, budgeting... If a user asks about unrelated topics (coding, politics, trivia, medical), politely decline."*
- **Anti-Jailbreak Protection:** Instructions explicitly command the model to ignore user attempts to override system prompts, change personas, or reveal system instructions.
- **Output Length & Style Control:** Enforces concise 3-4 bullet point maximums and prohibits emojis to maintain corporate/professional presentation.

Q18: How does your code handle API key security and SDK version mismatches?

Answer / Key Points:

- **Environment Variables:** API keys are loaded via **python-dotenv** (**os.environ.get("GEMINI_API_KEY")**), ensuring credentials are never hardcoded in source control.
- **Multi-Layer Fallback in call_gemini_chat:**

1. Attempts to use the official **google.generativeai** Python SDK.
2. If the SDK is missing or fails, falls back to direct REST API calls using standard Python **urllib.request**.
3. If the REST API returns HTTP 404 (model not found for **gemini-2.5-flash**), it automatically downgrades and retries with **gemini-1.5-flash**, ensuring high availability and resilience across different Google Cloud account tiers.

Section 6: Scalability, Edge Cases & System Evolution

Q19: What are the current technical limitations of SpendWise, and how would you architect it for 100,000 active users?

Answer / Key Points:

- **Current Limitations:** Local filesystem storage (**outputs/*.json**), synchronous blocking OCR/LLM calls, and single-tenant Streamlit memory.
- **Enterprise Architecture for 100k Users:**
- **Database:** Migrate JSON file storage to a relational database (PostgreSQL) with tables for **Users**, **Receipts**, **LineItems**, and **AuditReports**. Use indexed JSONB columns for flexible LLM metadata.
- **Async Processing:** Use an S3/GCS bucket for receipt image uploads. Trigger AWS Lambda / Celery workers via RabbitMQ/SQS to run OCR and CrewAI pipelines asynchronously.
- **Frontend:** Replace Streamlit with a React/Next.js frontend communicating with a FastAPI or Node.js backend via REST/GraphQL and WebSockets for real-time audit progress updates.

Q20: How would you handle edge cases like multi-language receipts, handwritten bills, or faded ink?

Answer / Key Points:

- **Multi-Language:** Integrate vision-language models (like Gemini Pro Vision or GPT-4o) directly on receipt images, which naturally translate and extract item names across dozens of languages without needing specialized OCR language packs.

- **Image Pre-processing Pipeline:** Before sending images to OCR, pass them through an OpenCV preprocessing step: grayscale conversion, adaptive thresholding (to handle shadows/faded ink), deskewing (to fix tilted camera angles), and contrast enhancement.
- **Confidence Scoring:** Modify the OCR ingestion to output confidence scores per line item. Items below 70% confidence could be flagged in the UI with a yellow badge for manual user review.

Q21: How would you implement automated testing and CI/CD for an AI-agent application like SpendWise?

Answer / Key Points:

- **Deterministic Unit Tests:** Test helper functions (**clean_json_response**, **transform_receipt_payload**, math aggregators) using standard **pytest** assertions.
- **LLM Evaluation (LLMOps / Eval Pipelines):**
- Create a "Golden Dataset" of 50 standardized receipt OCR outputs with known correct categorizations.
- In CI/CD (GitHub Actions), run the Categorizer Agent against the golden dataset and measure **Classification Accuracy** (must exceed 90% to pass build).
- Use an LLM-as-a-Judge approach to evaluate the Advisor Agent's output for tone, relevance, and adherence to safety guardrails.

Section 7: Behavioral & Project Storytelling (HR / Managerial Round)

Q22: What inspired you to build SpendWise AI Studio, and why target college students?

Talking Points:

- *"During college, my friends and I noticed that standard budgeting apps like Mint or Splitwise only track total transaction amounts (e.g., '$45 at Walmart'). But at Walmart, a student might buy groceries, notebooks for class, and a video game. Without item-level visibility, budgeting advice is generic and useless."*
- *"I wanted to combine cutting-edge OCR and autonomous AI agents to solve this micro-budgeting problem, making financial literacy interactive, beautiful, and tailored to student lifestyles."*

Q23: What was the most challenging technical obstacle you overcame while developing this project?

Talking Points:

- *"The biggest hurdle was bridging the gap between messy, unpredictable OCR text and structured, deterministic accounting analysis."*
- *"Initially, sending raw receipt text to the LLM caused hallucinations—it would invent prices or miscalculate totals. I solved this by strictly separating responsibilities: using OCR table mode (**isTable=True**) to extract clean price-item pairs, using deterministic Python loops for all mathematical sums, and restricting the AI agents purely to semantic categorization and high-level strategy."*
- *"Another challenge was styling Streamlit to look like a premium SaaS product, which I solved by writing custom CSS injections and designing a floating glassmorphic chat widget."*

Q24: If you had two more weeks to expand SpendWise, what feature would you engineer next?

Talking Points:

- **Real-Time Bank & UPI Sync:** Integrating open banking APIs (like Plaid or Setu UPI) to automatically match scanned receipt totals against actual digital bank transactions.
- **Predictive Budget Alerting:** Using time-series forecasting (ARIMA or Prophet) on historical line items to predict when a student is on track to overspend on food or entertainment before the month ends.
- **Peer Benchmarking:** Anonymized, opt-in comparative analytics allowing students to see how their spending distribution compares to the average student on their campus.

Quick Reference: SpendWise Codebase Mapping

When answering questions, point directly to specific files and functions to demonstrate deep technical mastery:

- **app.py**: Streamlit UI, **apply_custom_styles()** (CSS), **render_navbar()**, **render_kpi_section()**, **call_gemini_chat()** (REST API fallback & safety guardrails).
- **run.py**: CrewAI pipeline orchestration, **clean_json_response()** (regex JSON parser), batch processing loop (**batch_size = 20**), deterministic math calculations.
- **agents_And_tasks.py**: Definitions of **categorizer_agent()**, **analyst_agent()**, **advisor_agent()**, and their structured prompt tasks.
- **scan-reciepts-hf.py**: **ReceiptCaptureService**, Asprise OCR HTTP POST requests, rate limit 429 retry loops, **transform_receipt_payload()**.
- **ocr_space_receipts.py**: **extract_receipt_lines()** (**isTable=True**, **OCREngine=2**), procedural keyword matching against **CATEGORIES**.
- **analysis.py**: CLI terminal summary dashboard printing verified expenditure and coaching tips.